

**ESTUDO DE CASO
RAÍZES DO NORDESTE**

1 INTRODUÇÃO.....	4
2 ANÁLISE DE REQUISITOS.....	5
2.1 Requisitos Funcionais.....	5
2.2 Requisitos Não Funcionais.....	6
3 MODELAGEM E ARQUITETURA.....	7
3.1 Diagramas.....	7
3.1.1 Diagrama de caso de uso.....	7
3.1.1.1 Descrição dos principais casos de uso.....	8
3.1.2 Diagrama Entidade-Relacionamento.....	12
3.1.3 Diagrama de Classes.....	13
3.2 Arquitetura.....	13
3.2.1 Camada de Domínio (Domain).....	14
3.2.2 Camada de Aplicação (Application).....	14
3.2.3 Camada de Infraestrutura (Infrastructure).....	14
3.2.4 Camada de API (Controllers).....	14
4 API E ENDPOINTS.....	15
4.1 /auth - Módulo de autenticação.....	15
4.2 /pedidos - Módulo de pedidos (fluxo crítico).....	17
4.3 /estoque - Módulo de movimentação do estoque.....	24
4.4 /unidades - Módulo de unidades.....	29
4.5 /pagamentos - Módulo de pagamentos.....	32
4.6 /cardapio - Módulo de itens do cardápio.....	33
4.7 /produtos - Módulo de produtos.....	39
4.8 /auditorias - Módulo de auditoria de ações sensíveis.....	42
5 ENTREGA TÉCNICA.....	44
6 PLANOS DE TESTES E EVIDÊNCIAS.....	44
7 CONCLUSÃO.....	45
8 REFERÊNCIAS.....	48

1 INTRODUÇÃO

A expansão do restaurante Raízes do Nordeste para um modelo de franquias com múltiplas unidades torna necessária a adoção de uma solução back-end centralizada, capaz de garantir consistência operacional, escalabilidade e integração entre canais de atendimento. Nesse contexto, foi desenvolvida uma API RESTful responsável por centralizar as regras de negócio do sistema, permitindo a interoperabilidade entre diferentes interfaces de consumo.

A multicanalidade é tratada como requisito estrutural do domínio, sendo implementada por meio do registro do canal de origem de cada pedido (`canalPedido`), possibilitando rastreabilidade, análise de comportamento e consistência das operações independentemente da interface utilizada (web, mobile ou integrações externas). A API opera de forma stateless, garantindo escalabilidade horizontal e independência entre requisições.

O sistema contempla os principais requisitos funcionais do back-end, incluindo gestão de pedidos, controle de estoque por unidade, administração de cardápio, fidelização de clientes e controle de unidades franqueadas. Também são implementados mecanismos de autenticação e autorização baseados em perfis de acesso (CLIENTE e ADMIN), garantindo segregação de responsabilidades e segurança das operações. A relevância do sistema está na capacidade de garantir escalabilidade, rastreabilidade e controle centralizado das operações, reduzindo inconsistências entre unidades, aumentando a eficiência do atendimento e garantindo a aplicabilidade da LGPD e finalidade de todos os dados coletados

Em relação à LGPD, o sistema adota princípios de finalidade e minimização de dados, através de Hash de senhas, e consentimento explícito do usuário para utilização de dados, com registro de logs de operações críticas para rastreabilidade, controle de acesso aos dados sensíveis e estrutura preparada para políticas de

retenção e auditoria, garantindo conformidade com requisitos básicos de privacidade.

Como requisito de integração externa, o sistema prevê o uso de um gateway de pagamento simulado (mock), permitindo a validação do fluxo de pedidos com processamento de transações sem dependência de serviços reais. Isso garante desacoplamento e facilita testes e evolução futura.

Por fim, o projeto adota uma estratégia incremental de desenvolvimento, priorizando um MVP composto pelos módulos essenciais de autenticação, pedidos, estoque e cardápio, enquanto funcionalidades futuras incluem ampliação do programa de fidelidade, integração com novos meios de pagamento e expansão de perfis de usuários para maior granularidade operacional.

2 ANÁLISE DE REQUISITOS

2.1 Requisitos Funcionais

Quadro 1 - Requisitos funcionais.

id	Descrição
RF01	Cadastro e autenticação de usuários
RF02	Visualização/consulta de cardápio por unidade
RF03	Realização de pedidos
RF04	Atualização do status do pedido (cozinha → pronto → entregue / cancelado)
RF05	Controle de estoque
RF06	Programa de fidelização (pontos e resgate simples, com consentimento)
RF07	Promoções/campanhas
RF08	Solicitação de pagamento via serviço externo
RF09	Múltiplos canais (APP, TOTEM, WEBSITE, PICKUP, BALCAO)

Fonte: Autor (2026).

2.2 Requisitos Não Funcionais

Quadro 2 - Requisitos não funcionais.

id	Descrição
RNF01	Consonância com a LGPD
RNF02	Senha com Hash
RNF03	Logs e Auditorias de ações sensíveis
RNF04	Tolerância a falhas no pagamento
RNF05	Disponibilidade do sistema
RNF06	Documentação
RNF07	Desempenho em horários de pico
RNF08	Utilização de Token

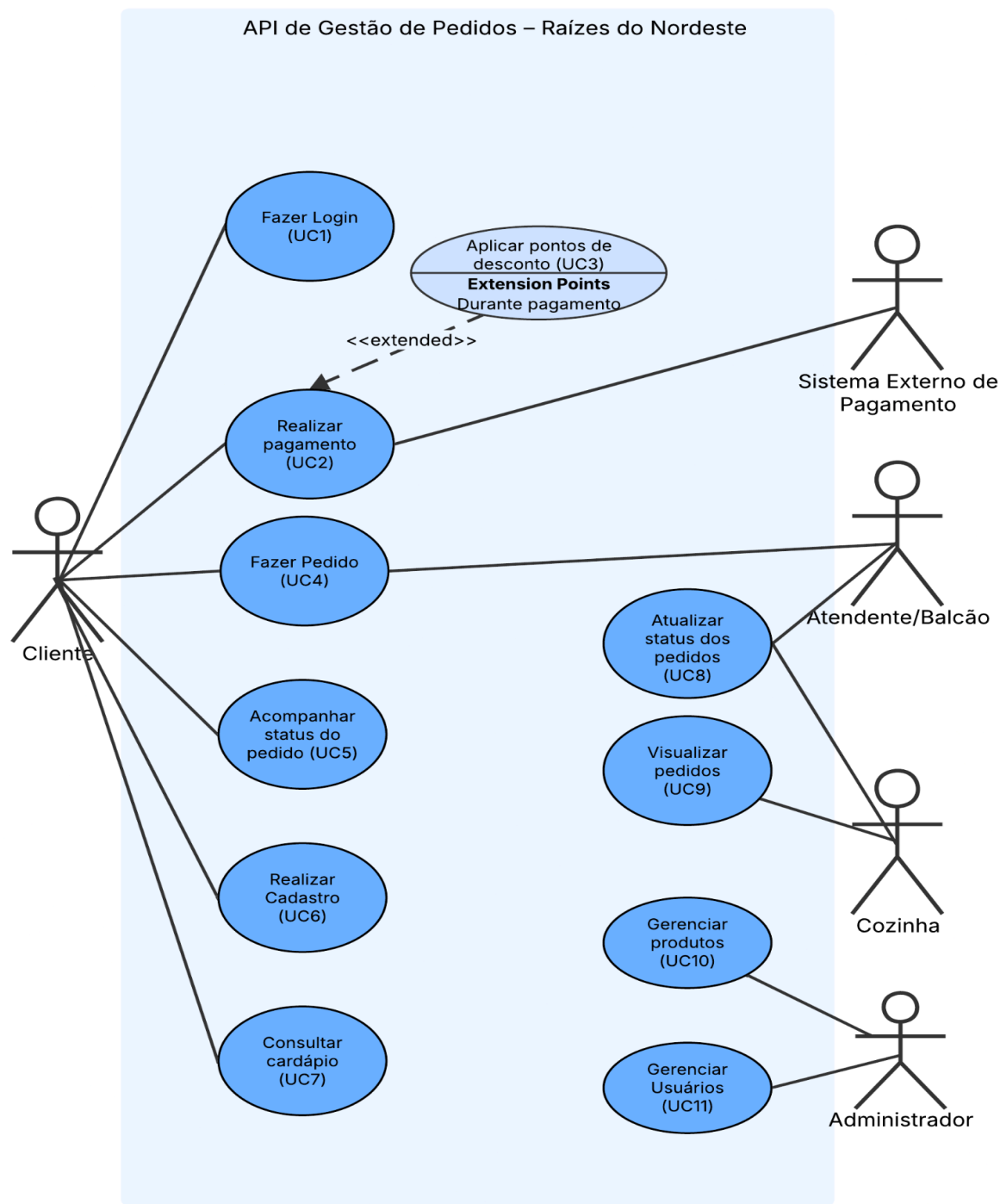
Fonte: Autor (2026).

3 MODELAGEM E ARQUITETURA

3.1 Diagramas

3.1.1 Diagrama de caso de uso

Figura 1 - Diagrama de caso de uso.



3.1.1.1 Descrição dos principais casos de uso

Quadro 3 - Descrição do UC 1.

UC 1 – Nome	Fazer Login.
Descrição	O cliente faz o login no sistema.
Ator principal	Cliente.
Demais atores	N/C.
Pré-condições	Usuário previamente cadastrado no sistema.
Pós-condições	1. Usuário autenticado com sucesso. 2. Token de acesso gerado.
Origem das informações	• Banco de dados do sistema. • Usuário.
Usuários responsáveis	N/C.
Pendências	N/C.
Fluxo base	1. Usuário informa credenciais (email e senha). 2. Sistema valida as credenciais. 3. Sistema gera token de autenticação. 4. Usuário acessa o sistema.
Fluxo alternativo	• Credenciais inválidas - sistema nega acesso. • Usuário não encontrado - sistema retorna erro.
Regras de negócio	RN01: Usuário deve possuir cadastro ativo. RN02: Senha deve ser validada com hash seguro.

Fonte: Autor (2026).

Quadro 4 - Descrição do UC 2.

UC 2 – Nome	Realizar pagamento
Descrição	Permite processar o pagamento de um pedido por meio de um sistema externo.
Ator principal	Cliente.
Demais atores	Sistema externo de pagamento.
Pré-condições	1. Pedido criado. 2. Pagamento com status “PENDENTE”
Pós-condições	Pagamento atualizado para “CONFIRMADO”, “CANCELADO” ou “NEGADO”.
Origem das informações	• Cliente • Sistema interno • Sistema externo de pagamento
Usuários responsáveis	N/C.
Pendências	N/C.
Fluxo base	1. Cliente escolhe forma de pagamento. 2. Sistema externo processa a transação. 3. Sistema recebe o resultado. 4. Sistema atualiza o status do pedido.
Fluxo alternativo	• Pagamento recusado - status atualizado para “CANCELADO”. • Falha de comunicação - sistema retorna erro.
Regras de negócio	RN01: Pedido deve estar aguardando pagamento. RN02: Valor do pedido deve ser válido.

Fonte: Autor (2026).

Quadro 5 - Descrição do UC 3.

UC 3 – Nome	Aplicar pontos de desconto
Descrição	Permite ao cliente utilizar pontos acumulados para obter desconto no pedido.
Ator principal	Cliente.
Demais atores	N/C.
Pré-condições	1. Cliente autenticado. 2. Pedido criado. 3. Cliente possui pontos disponíveis.
Pós-condições	1. Valor do pedido atualizado com desconto. 2. Pontos debitados do cliente.
Origem das informações	• Cliente • Sistema interno
Usuários responsáveis	N/C.
Pendências	Definição da regra de conversão de pontos em valor.
Fluxo base	1. Cliente opta por utilizar pontos. 2. Sistema verifica saldo disponível. 3. Sistema valida regras de uso. 4. Sistema aplica desconto ao pedido. 5. Sistema atualiza saldo de pontos.
Fluxo alternativo	• Pontos insuficientes - sistema não aplica desconto. • Pontos expirados - sistema bloqueia uso.
Regras de negócio	RN01: Pontos só podem ser usados se houver saldo suficiente. RN02: Pontos podem ter validade. RN03: Desconto não pode exceder valor do pedido.

Fonte: Autor (2026).

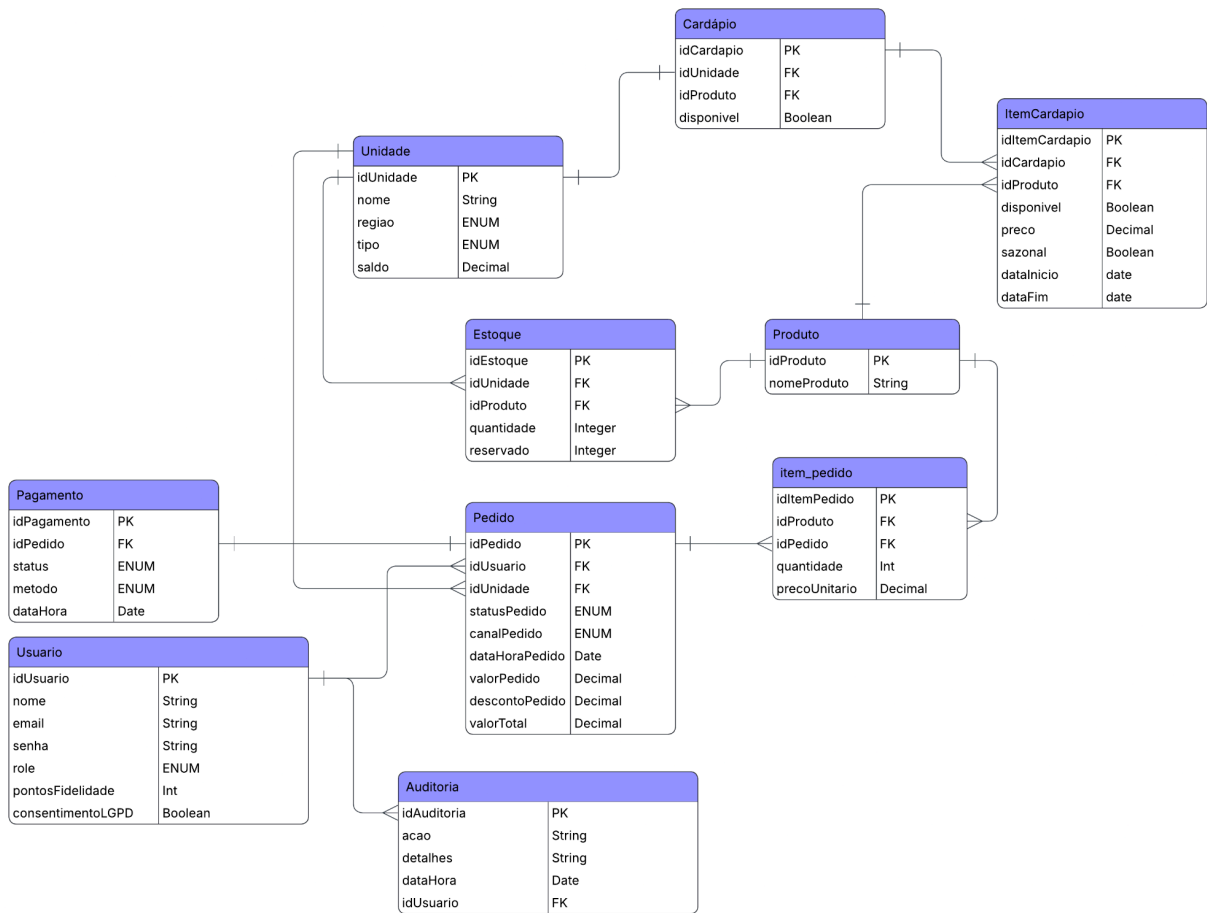
Quadro 6 - Descrição do UC 4.

UC 4 – Nome	Fazer pedido
Descrição	Permite ao cliente criar um pedido com base nos produtos disponíveis.
Ator principal	Cliente
Demais atores	Atendente
Pré-condições	Produtos cadastrados no sistema.
Pós-condições	Pedido criado com status de pagamento “PENDENTE” e status do pedido “COZINHA”.
Origem das informações	1. Cliente 2. Sistema interno
Usuários responsáveis	N/C.
Pendências	N/C.
Fluxo principal	1. Cliente seleciona produtos do cardápio. 2. Sistema valida disponibilidade em estoque. 3. Sistema calcula valor total. 4. Sistema registra o pedido. 5. Sistema define status inicial.
Fluxo alternativo	Produto indisponível - sistema impede inclusão.
Regras de negócio	RN01: Pedido deve conter ao menos um item. RN02: Sistema deve validar estoque antes de confirmar. RN03: Pedido deve registrar o canal (app, web, totem).

Fonte: Autor (2026).

3.1.2 Diagrama Entidade-Relacionamento

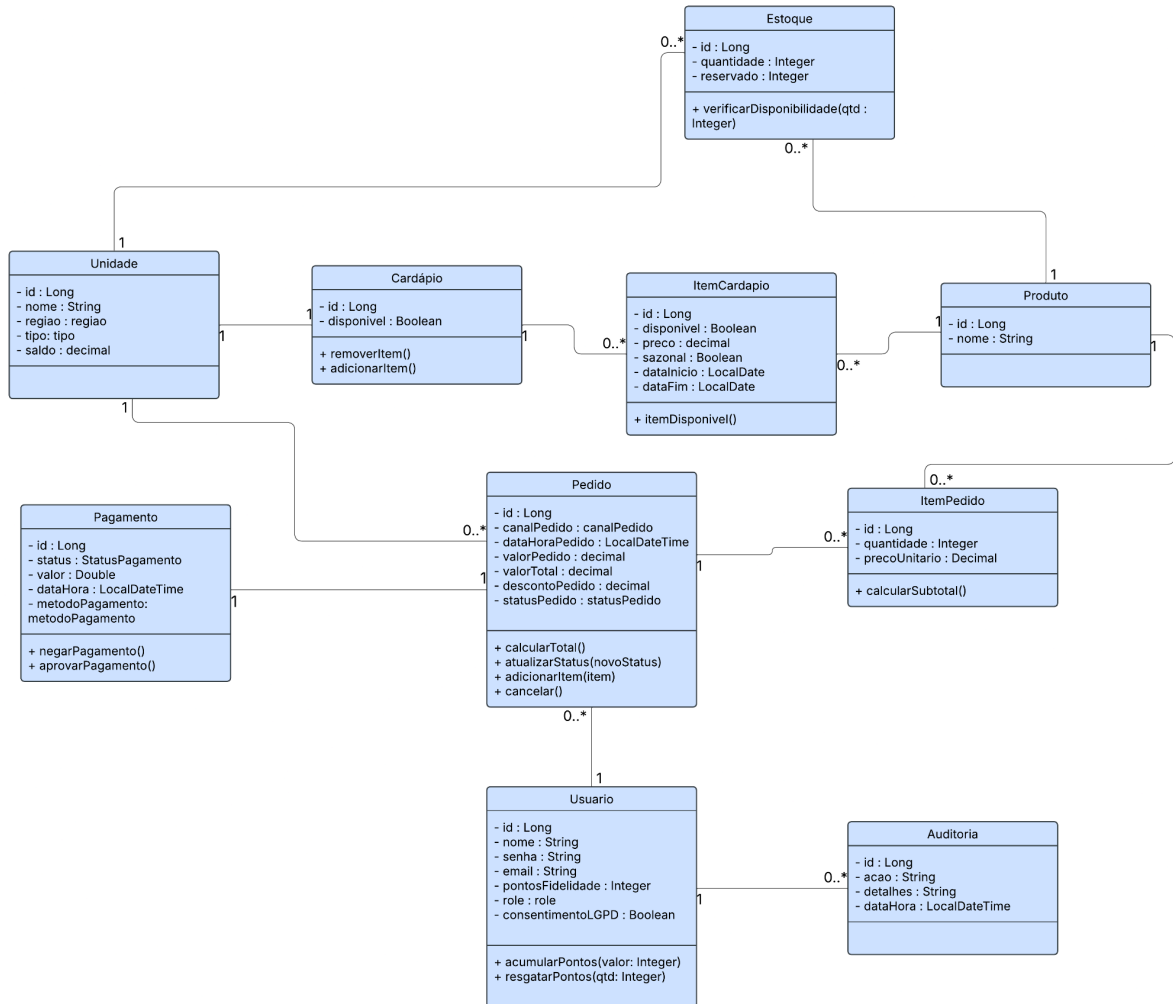
Figura 2 - DER (Diagrama Entidade-Relacionamento) - Modelo de Dados.



Fonte: Autor (2026).

3.1.3 Diagrama de Classes

Figura 3 - Diagrama de Classes



Fonte: Autor (2026).

3.2 Arquitetura

A solução foi desenvolvida seguindo uma arquitetura em camadas, com o objetivo de promover a separação de responsabilidades, facilitar a manutenção do código e permitir escalabilidade da aplicação. Essa abordagem permite o isolamento das regras de negócio em relação aos detalhes de infraestrutura e interface, reduzindo o acoplamento entre os componentes.

3.2.1 Camada de Domínio (Domain)

A camada de domínio é responsável por concentrar as entidades centrais do sistema e suas regras de negócio, como cálculo de valores de pedidos, controle de status e validações relacionadas às regras operacionais do sistema.

3.2.2 Camada de Aplicação (Application)

A camada de aplicação contém os serviços responsáveis por orquestrar os casos de uso do sistema, coordenando o fluxo entre domínio e infraestrutura.

Nesta camada estão serviços como PedidoService, AutenticacaoService, UnidadeService e AuditoriaService, responsáveis por operações como criação de pedidos, autenticação de usuários, aplicação de regras de negócio e registro de auditoria.

Essa camada também realiza validações de fluxo, como verificação de estoque, aplicação de regras de fidelização e controle de status de pedidos.

3.2.3 Camada de Infraestrutura (Infrastructure)

A camada de infraestrutura é responsável pelos detalhes técnicos da aplicação, incluindo persistência de dados, integração com bibliotecas externas e serviços auxiliares.

Nesse projeto, essa camada inclui os repositórios JPA responsáveis pelo acesso ao banco de dados, além de componentes como o JwtService, responsável pela geração e validação dos tokens de autenticação.

Também são incluídas configurações do Spring Security, responsáveis pelo controle de autenticação e autorização dos usuários.

3.2.4 Camada de API (Controllers)

A camada de API é responsável pela exposição dos endpoints REST, recebendo requisições externas e retornando respostas padronizadas.

Os controllers definem as rotas da aplicação, realiza, o mapeamento de requests e responses e aplicam regras de autorização através de anotações.

Essa camada também integra a autenticação via Spring Security, utilizando o contexto de usuário autenticado para controlar o acesso a recursos sensíveis.

3.2.5 Justificativa da implementação da Arquitetura

A adoção dessa arquitetura permite uma clara separação de camadas e responsabilidades, reduzindo o acoplamento entre componentes do sistema. Isso facilita a manutenção, testes e evolução do projeto. Além disso, essa organização favorece a escalabilidade, pois novas funções podem ser adicionadas sem comprometer a estrutura existente.

4 API E ENDPOINTS

4.1 /auth - Módulo de autenticação

/auth - Cadastrar

Finalidade: Permitir o cadastro de novos clientes no sistema.

Rota: /auth/register

Método: POST

Autenticação e permissões: Não requer autenticação.

Parâmetros: Não possui.

Request:

```
{  
  
  "nome": "João Silva",  
  "email": "joao@gmail.com",  
  "senha": "123456",  
  "consentimento": true  
}
```

Response: (201 Created)

```
{
```

```
"id": 1,  
"nome": "João Silva",  
"email": "joao@gmail.com",  
"pontosFidelidade": 0,  
"role": "CLIENTE"  
}
```

Status code: 201, 401, 400.

Erro 400 (exemplo):

```
{  
  "timestamp": "2026-06-04T13:56:08.4769791",  
  "status": 400,  
  "erro": "Erro de validação",  
  "mensagens": [  
    "nome: não deve estar em branco",  
    "email: deve ser um endereço de e-mail bem formado"  
  ],  
  "path": "/auth/register"  
}
```

/auth - Logar

Rota: /auth/login

Finalidade: Autenticar um usuário e gerar um token JWT para acesso aos recursos protegidos da API.

Método: POST

Autenticação e permissões: Não requer autenticação.

Parâmetros: Não possui query params ou path params.

Request:

```
{  
  "email": "joao@gmail.com",  
  "senha": "123456"  
}
```

Response (200):

```
{
```


"token": "eyJhbGciOiJIUzI1Ni..."

}

Status code: 200, 401.

Erro 401:

{

"timestamp": "2026-06-05T20:06:44.4120828",

"status": 401,

"erro": "Não autorizado",

"mensagem": "Credenciais inválidas",

"path": "/auth/login"

}

4.2 /pedidos - Módulo de pedidos (fluxo crítico)

/pedido - Criar pedido

Rota: /pedidos

Finalidade: Realizar criação de pedido.

Método: POST

Autenticação e permissões: JWT, perfil CLIENTE.

Request:

{

"unidadeId": 1,

"canalPedido": "APP",

"metodoPagamento": "DEBITO",

"itens": [

{

"produtoId": 2,

"quantidade": 1

}

]

}

Response (201):

```
{
  "id": 1,
  "statusPedido": "COZINHA",
  "canalPedido": "APP",
  "horaPedido": "2026-06-04T14:49:19.2462121",
  "valorPedido": 8.99,
  "descontoPedido": 0,
  "valorTotal": 8.99,
  "unidadeId": 1,
  "usuario": {
    "id": 1,
    "nome": "Joao da Silva",
    "email": "joao@gmail.com",
    "pontosFidelidade": 17,
    "role": "CLIENTE"
  },
  "itens": [
    {
      "produtoId": 2,
      "nomeProduto": "Cuscuz",
      "quantidade": 1,
      "precoUnitario": 8.99
    }
  ]
}
```

Status code:

- 201 criado
- 400 Json inválido

- 401/403 sem permissão
- 404 unidade/estoque inexistente
- 409 conflito com regra de negócio

Erro 404:

```
{
  "timestamp": "2026-06-05T20:30:14.1660577",
  "status": 404,
  "erro": "Recurso não encontrado",
  "mensagem": "Produto sem estoque",
  "path": "/pedidos"
}
```

/pedidos - Consultar status

Finalidade: Consultar o status atual do pedido e do pagamento.

Método: GET

Rota: /pedidos/{id}/status

Autenticação e permissões: JWT obrigatório. Perfis ADMIN e CLIENTE. O cliente pode consultar apenas os próprios pedidos. O administrador pode consultar qualquer pedido.

Parâmetros: {id} (Path) - Identificador do pedido.

Request: Não possui corpo da requisição.

Response (200):

```
{
  "id": 1,
  "statusPedido": "PRONTO",
  "statusPagamento": "CONFIRMADO"
}
```

Status code:

- 200 Status consultado com sucesso.
- 403 Sem permissão
- 404 Pedido não encontrado

Erro (404):

```
{
```

```
"timestamp": "2026-06-05T21:34:47.0960183",  
"status": 404,  
"erro": "Recurso não encontrado",  
"mensagem": "Pedido não encontrado",  
"path": "/pedidos/1111"  
}
```

/auth - Listar pedidos

Finalidade: Listar os pedidos cadastrados no sistema, permitindo filtragem por canal de pedido.

Método: GET

Rota: /pedidos

Autenticação e permissões: JWT obrigatório. Perfil ADMIN.

Parâmetros:

Query:

- page - Número da página
- size - Quantidade de registros
- sort - Ordenação dos resultados
- canalPedido - Filtra pedidos por canal

Exemplo de requisição: GET /pedidos?canalPedido=TOTEM

Response (200):{

```
"content": [  
  {  
    "id": 5,  
    "statusPedido": "CANCELADO",  
    "canalPedido": "TOTEM",  
    "horaPedido": "2026-06-02T18:00:40.870397",  
    "valorPedido": 8.99,  
    "descontoPedido": 0.00,  
    "valorTotal": 8.99,  
    "unidadeId": 1,  
    "usuario": {  
      "id": 5,
```

```
        "nome": "Manu da Silva",
        "email": "manu@gmail.com",
        "pontosFidelidade": 17,
        "role": "CLIENTE"
    },
    "itens": [
        {
            "produtoid": 3,
            "nomeProduto": "Bolo de mandioca",
            "quantidade": 1,
            "precoUnitario": 8.99
        }
    ]
},
"pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {
        "empty": true,
        "sorted": false,
        "unsorted": true
    },
    "offset": 0,
    "paged": true,
    "unpaged": false
},
"last": true,
"totalElements": 1,
"totalPages": 1,
"size": 20,
"number": 0,
"sort": {
    "empty": true,
```

```
        "sorted": false,
        "unsorted": true
    },
    "first": true,
    "numberOfElements": 1,
    "empty": false
}
```

Status code:

- 200 Lista retornada com sucesso
- 400 Valor inválido para canalPedido
- 403 Usuário sem permissão

Erro (400):{

```
    "timestamp": "2026-06-05T21:33:51.6898631",
    "status": 400,
    "erro": "JSON inválido",
    "mensagem": "Valor inválido enviado no corpo da requisição",
    "path": "/pedidos"
}
```

/pedidos - Buscar pedido

Finalidade: Consultar os dados completos de um pedido.

Método: GET

Rota: /pedidos/{id}

Autenticação e permissões: JWT obrigatório. Perfis ADMIN e CLIENTE. O cliente pode visualizar apenas os próprios pedidos. O administrador pode visualizar qualquer pedido.

Parâmetros: {id} (Path) - Identificador do pedido.

Request: Não possui corpo da requisição.

Response (200): {

```
    "id": 6,
    "statusPedido": "CANCELADO",
    "canalPedido": "APP",
    "horaPedido": "2026-06-04T14:35:26.045757",
    "valorPedido": 8.99,
```

```
"descontoPedido": 0.00,
"valorTotal": 8.99,
"unidadeId": 1,
"usuario": {
  "id": 1,
  "nome": "Joao da Silva",
  "email": "joao@gmail.com",
  "pontosFidelidade": 0,
  "role": "CLIENTE"
},
"itens": [
  {
    "produtoId": 2,
    "nomeProduto": "Cuscuz",
    "quantidade": 1,
    "precoUnitario": 8.99
  }
]
}
```

Status code:

- 200 Pedido encontrado
- 403 Usuário não pode acessar esse pedido
- 404 Pedido não encontrado

Erro (404): {

```
"timestamp": "2026-06-05T21:36:46.791564",
"status": 404,
"erro": "Recurso não encontrado",
"mensagem": "Pedido não encontrado",
"path": "/pedidos/1112"
}
```

/pedidos - Cancelar pedido

Finalidade: Permitir o cancelamento de um pedido existente.

Método: PUT

Rota: /pedidos/{id}/cancelamento

Autenticação e permissões: JWT obrigatório. Perfis ADMIN e CLIENTE. O cliente pode cancelar apenas os próprios pedidos. O administrador pode cancelar qualquer pedido.

Parâmetros: {id} (Path) - Identificador do pedido.

Request: Não possui corpo da requisição.

Response: Sem conteúdo.

Status code:

- 204 Pedido cancelado com sucesso.
- 403 Usuário sem permissão para cancelar o pedido.
- 404 Pedido não encontrado.
- 409 Pedido já cancelado ou regra de negócio violada.

Erro (409): {

```
"timestamp": "2026-06-05T21:45:47.3664125",  
"status": 409,  
"erro": "Erro de negócio",  
"mensagem": "Pedido já cancelado",  
"path": "/pedidos/8/cancelamento"
```

}

4.3 /estoque - Módulo de movimentação do estoque

Autenticação e permissões: JWT obrigatório para todos os endpoints. Apenas o perfil ADMIN pode acessar os métodos listados a seguir.

/estoque - Adicionar produto

Finalidade:: Adicionar um produto ao estoque de uma unidade específica.

Método: POST

Rota: /estoque/unidades/{unidadeId}/produtos/{produtoId}

Parâmetros:

{unidadeId} (Path) - Identificador da unidade.

{produtoId} (Path) - Identificador do produto.

Request:

{


```
"quantidade": 10
}
```

Response:

```
{
  "id": 1,
  "quantidade": 10,
  "unidadeId": 1,
  "produto": {
    "id": 2,
    "nome": "Cuscuz",
    "preco": 8.99
  }
}
```

Status code: 201, 400, 401, 403, 404, 409.

Erro (409): {

```
  "timestamp": "2026-06-05T21:48:38.9954557",
  "status": 409,
  "erro": "Erro de negócio",
  "mensagem": "Produto já existe no estoque",
  "path": "/estoque/unidades/1/produtos/1"
}
```

/estoques - Atualizar quantidade

Finalidade: Atualizar a quantidade disponível de um produto em uma unidade.

Método: PUT

Rota: /estoque/unidades/{unidadeId}/produtos/{produtoId}

Parâmetros:

{unidadeId} (Path) - Identificador da unidade.

{produtoId} (Path) - Identificador do produto.

Request:

```
{
  "quantidade": 25
}
```

Response:{

```
  "id": 1,
```

```
"quantidade": 25,  
"unidadeId": 1,  
"produto": {  
  "id": 2,  
  "nome": "Cuscuz",  
  "preco": 8.99  
}  
}
```

Status code: 200, 400, 401, 403, 404, 409.

Erro (409): {
 "timestamp": "2026-06-05T21:51:40.7980242",
 "status": 409,
 "erro": "Erro de negócio",
 "mensagem": "Quantidade não pode ser negativa",
 "path": "/estoque/unidades/1/produtos/7"
}

/estoque - Remover produto

Finalidade: Remover um produto do estoque de uma unidade.

Método: DELETE

Rota: /estoque/unidades/{unidadeId}/produtos/{produtoId}

Parâmetros:

{unidadeId} (Path) - Identificador da unidade.

{produtoId} (Path) - Identificador do produto.

Request: Não possui corpo da requisição.

Response: Sem conteúdo.

Status code: 204, 400, 401, 403, 404.

Erro (404):{
 "timestamp": "2026-06-05T22:30:17.0182516",
 "status": 404,
 "erro": "Recurso não encontrado",
 "mensagem": "Estoque não encontrado",
 "path": "/estoque/unidades/2/produtos/1"
}

/estoque - Listar

Finalidade: Listar todos os produtos disponíveis no estoque de uma unidade.

Método: GET

Rota: /estoque/unidades/{unidadeId}

Parâmetros:

{unidadeId} (Path) - Identificador da unidade.

{produtoId} (Path) - Identificador do produto.

Request: Não possui corpo da requisição.

Response: {

```
"content": [  
  {  
    "id": 1,  
    "quantidade": 0,  
    "unidadeId": 1,  
    "produto": {  
      "id": 1,  
      "nomeProduto": "Açaí"  
    }  
  },  
  {  
    "id": 2,  
    "quantidade": 1,  
    "unidadeId": 1,  
    "produto": {  
      "id": 2,  
      "nomeProduto": "Cuscuz"  
    }  
  },  
  {  
    "id": 3,  
    "quantidade": 0,  
    "unidadeId": 1,  
    "produto": {
```

```
        "id": 3,
        "nomeProduto": "Bolo de mandioca"
    }
},
{
    "id": 4,
    "quantidade": 3,
    "unidadeId": 1,
    "produto": {
        "id": 4,
        "nomeProduto": "Carne de sol"
    }
},
{
    "id": 5,
    "quantidade": 3,
    "unidadeId": 1,
    "produto": {
        "id": 7,
        "nomeProduto": "Buchada de bode"
    }
}
],
"pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {
        "empty": true,
        "sorted": false,
        "unsorted": true
    },
    "offset": 0,
    "paged": true,
    "unpaged": false
}
```

```
    },
    "last": true,
    "totalPages": 1,
    "totalElements": 5,
    "first": true,
    "numberOfElements": 5,
    "size": 20,
    "number": 0,
    "sort": {
        "empty": true,
        "sorted": false,
        "unsorted": true
    },
    "empty": false
}
```

Status code: 200, 401, 403.

Erro (401):{

```
    "timestamp": "2026-06-07T14:08:26.1819678",
    "status": 401,
    "erro": "Não autorizado",
    "mensagem": "Autenticação necessária para acessar este recurso",
    "path": "/estoque/unidades/1"
}
```

4.4 /unidades - Módulo de unidades

Autenticação e permissões: JWT obrigatório para todos os endpoints. Apenas o perfil ADMIN pode acessar os métodos listados a seguir.

/unidades - Criar unidade

Finalidade: Criar uma nova unidade no sistema (ex.: restaurante/filial).

Método: POST

Rota: /unidades

Parâmetros: Nenhum.

Request:{

"nome": "Unidade Centro",

"regiao": "NORDESTE",

"tipo": "COMPLETA"

}

Response:{

"id": 6,

"nome": "Unidade Centro",

"regiao": "NORDESTE",

"tipo": "COMPLETA",

"saldo": 0,

"cardapio": {

"id": 6,

"disponivel": true,

"itens": []

}

}

Status Code: 201, 400, 403.

Erro (400):{

"timestamp": "2026-06-05T22:33:11.5097698",

"status": 400,

"erro": "JSON inválido",

"mensagem": "Valor inválido enviado no corpo da requisição",

"path": "/unidades"

}

/unidades - Editar unidade

Finalidade: Atualizar dados de uma unidade existente .

Método: PUT

Rota: /unidades/{id}

Parâmetros: {id} (Path) - Identificador da unidade.

Request:{

"nome": "Unidade Principal Atualizada",

```
"regiao": "NORDESTE",  
"tipo": "COMPLETA"  
}
```

```
Response:{  
  "id": 6,  
  "nome": "Unidade Principal Atualizada",  
  "regiao": "NORDESTE",  
  "tipo": "COMPLETA",  
  "saldo": 0,  
  "cardapio": {  
    "id": 6,  
    "disponivel": true,  
    "itens": []  
  }  
}
```

Status code: 200, 400, 403, 404.

Erro (400):

```
{  
  "timestamp": "2026-06-05T22:35:27.8273023",  
  "status": 400,  
  "erro": "JSON inválido",  
  "mensagem": "Valor inválido enviado no corpo da requisição",  
  "path": "/unidades/2"  
}
```

/unidades - Listar unidades

Finalidade: Listar todas as unidades cadastradas (com paginação)

Método: GET

Rota: /unidades

Parâmetros:

{id} (Path) - Identificador da unidade

Query:

- page - Número da página
- size - Quantidade de registros

- sort - Ordenação dos resultados

Request: Não possui corpo da requisição.

Response:{

```
"content": [  
  {  
    "id": 1,  
    "nome": "Unidade Centro",  
    "regiao": "NORDESTE",  
    "tipo": "RESTAURANTE",  
    "saldo": 0,  
    "cardapio": {  
      "id": 1,  
      "itens": []  
    }  
  }  
],  
"pageable": {  
  "pageNumber": 0,  
  "pageSize": 10  
},  
"totalElements": 1,  
"totalPages": 1  
}
```

Status code: 200, 401, 403.

Erro (401):

```
{  
  "timestamp": "2026-06-06T14:36:51.9796836",  
  "status": 401,  
  "erro": "Não autorizado",  
  "mensagem": "Autenticação necessária para acessar este recurso",  
  "path": "/unidades"  
}
```


4.5 /pagamentos - Módulo de pagamentos

/pagamentos - Processar pagamento (simulação/mock)

Finalidade: Processar o pagamento de um pedido, aprovando ou recusando a transação.

Método: POST

Rota: /pagamentos/pedido/{pedidoId}

Autenticação e permissões: JWT obrigatório. Apenas o papel ADMIN.

Parâmetros: {id} (Path) - Identificador do pedido.

Request:{

“aprovado”: true

}

Response:{

“id”: 1,

“statusPagamento”: “CONFIRMADO”

}

Status code: 200, 401, 403, 409

Erro (409):

{

“timestamp”: “2026-06-05T22:42:11.6842841”,

“status”: 409,

“erro”: “Erro de negócio”,

“mensagem”: “Não é possível processar pagamento de um pedido cancelado”,

“path”: “/pagamentos/pedido/8”

}

4.6 /cardapio - Módulo de itens do cardápio

/cardapio - Adicionar item

Finalidade: Adicionar um produto ao cardápio de uma unidade, definindo preço, disponibilidade e sazonalidade.

Método: POST

Rota: /unidades/{unidadeId}/cardapio/produtos/{produtoId}

Autenticação e permissões: JWT obrigatório. Perfil ADMIN apenas pode adicionar produto a determinado cardápio.

Parâmetros: {unidadeId} (Path) - Identificador da unidade.

{produtoId} (Path) - Identificador do produto.

Request: {

```
"sazonal": true,  
"dataInicio": "2026-06-01",  
"dataFim": "2026-08-31",  
"preco": 14.90,  
"disponivel": true
```

}

Response: {

```
"id": 1,  
"preco": 14.90,  
"disponivel": true,  
"sazonal": true,  
"dataInicio": "2026-06-01",  
"dataFim": "2026-08-31",  
"produto": {  
  "id": 2,  
  "nome": "Cuscuz Nordestino"
```

```
}
```

}

Status code: 201, 400, 403, 404, 409.

Erro (409): {

```
"timestamp": "2026-06-05T22:45:08.6660579",  
"status": 409,  
"erro": "Erro de negócio",  
"mensagem": "Produto já existe no cardápio",  
"path": "/unidades/1/cardapio/produtos/8"
```

}

/cardapio - Listar itens

Finalidade: Listar os produtos disponíveis no cardápio de uma unidade.

Método: GET

Rota: /unidades/{unidadeId}/cardapio

Autenticação e permissões: JWT obrigatório. Papel CLIENTE ou ADMIN.

Parâmetros: {unidadeId} (Path) - Identificador da unidade.

Query:

- page - Número da página
- size - Quantidade de registros
- sort - Ordenação dos resultados

Request: Não possui corpo de requisição.

Response: {

```
"content": [  
  {  
    "id": 1,  
    "preco": 8.99,  
    "disponivel": true,  
    "sazonal": false,  
    "dataInicio": null,  
    "dataFim": null,  
    "produto": {  
      "id": 1,  
      "nomeProduto": "Açaí"  
    }  
  },  
  {  
    "id": 2,  
    "preco": 8.99,  
    "disponivel": true,  
    "sazonal": false,  
    "dataInicio": null,  
    "dataFim": null,  
    "produto": {  
      "id": 2,  
      "nomeProduto": "Cuscuz"    }  
  }  
]
```

```
    }
  },
  {
    "id": 3,
    "preco": 8.99,
    "disponivel": true,
    "sazonal": false,
    "dataInicio": null,
    "dataFim": null,
    "produto": {
      "id": 3,
      "nomeProduto": "Bolo de mandioca"
    }
  },
  {
    "id": 4,
    "preco": 8.99,
    "disponivel": false,
    "sazonal": false,
    "dataInicio": null,
    "dataFim": null,
    "produto": {
      "id": 4,
      "nomeProduto": "Carne de sol"
    }
  },
],
"pageable": {
  "pageNumber": 0,
  "pageSize": 20,
  "sort": {
    "empty": true,
    "sorted": false,
    "unsorted": true
  }
}
```

```
    },
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "totalElements": 4,
  "totalPages": 1,
  "last": true,
  "size": 20,
  "number": 0,
  "sort": {
    "empty": true,
    "sorted": false,
    "unsorted": true
  },
  "numberOfElements": 4,
  "first": true,
  "empty": false
}
```

Status code: 200, 403, 404.

Erro (404):{

```
  "timestamp": "2026-06-05T22:48:31.06851",
  "status": 404,
  "erro": "Recurso não encontrado",
  "mensagem": "Unidade não encontrada",
  "path": "/unidades/55/cardapio"
```

}

/cardapio - Editar item

Finalidade: Atualizar informações de um item do cardápio.

Método: PUT.

Rota: /unidades/{unidadeId}/cardapio/{itemId}

Autenticação e permissões: JWT obrigatório. Papel ADMIN apenas.

Parâmetros: {unidadeId} (Path) - Identificador da unidade.

{itemId} (Path) - Identificador do item do cardápio.

Request: {

```
"preco": 16.90,  
"disponivel": true,  
"dataInicio": "2026-06-01",  
"dataFim": "2026-08-31",  
"sazonal": true
```

}

Response: {

```
"id": 1,  
"preco": 16.90,  
"disponivel": true,  
"sazonal": true,  
"dataInicio": "2026-06-01",  
"dataFim": "2026-08-31",  
"produto": {  
  "id": 2,  
  "nome": "Cuscuz Nordestino"
```

```
}
```

}

Status code: 200, 400, 403, 404, 409.

Erro (409): {

```
"timestamp": "2026-06-05T22:50:57.5280129",  
"status": 409,  
"erro": "Erro de negócio",  
"mensagem": "Data início não pode ser maior que data fim",  
"path": "/unidades/1/cardapio/8"
```

}

/cardapio - Deletar item

Finalidade: Remover um produto do cardápio de uma unidade.

Método: DELETE.

Rota: /unidades/{unidadeId}/cardapio/{itemId}

Autenticação e permissões: JWT obrigatório; papel ADMIN.

Parâmetros: {unidadelId} (Path) - Identificador da unidade

{itemlId} (Path) - Identificador do item cardápio

Request: Não possui corpo de requisição.

Response: Sem conteúdo.

Status code: 204, 400, 403, 404, 409.

Erro (409): {

"timestamp": "2026-06-05T23:06:02.9843832",

"status": 409,

"erro": "Erro de negócio",

"mensagem": "Esse item não pertence a esse cardápio",

"path": "/unidades/6/cardapio/7"

}

4.7 /produtos - Módulo de produtos

Autenticação e permissões: JWT obrigatório para todos os endpoints. Apenas o perfil ADMIN pode acessar os métodos listados a seguir.

/produtos - Criar produto

Finalidade: Cadastrar um novo produto no sistema.

Método: POST.

Rota: /produtos

Parâmetros: Não possui.

Request: {

"nome": "Cuscuz Nordestino"

}

Response:{

"id": 1,

"nomeProduto": "Cuscuz Nordestino"

}

Status code: 201, 400, 403.

Erro (400): {

"timestamp": "2026-06-05T22:59:52.7478193",

```
{
  "status": 400,
  "erro": "Erro de validação",
  "mensagens": [
    "nome: não deve estar em branco"
  ],
  "path": "/produtos"
}
```

/produtos - Editar produto

Finalidade: Atualizar as informações de um produto existente.

Método: PUT

Rota: /produtos/{id}

Parâmetros: {id} (Path) - Identificador do produto.

Request:{

"nome": "Cuscuz Nordestino Especial"

}

Response:

{

"id": 1,

"nomeProduto": "Cuscuz Nordestino Especial"

}

Status code: 200, 400, 403, 404.

Erro (400): {

"timestamp": "2026-06-05T23:02:04.4039483",

"status": 400,

"erro": "Erro de validação",

"mensagens": [

"nome: não deve estar em branco"

],

"path": "/produtos/7"

}

/produtos - Listar produtos

Finalidade: Listar todos os produtos cadastrados.

Parâmetros:

Query:

- page - Número da página
- size - Quantidade de registros
- sort - Ordenação dos resultados

Request: Não possui corpo de requisição.

Response: {

```
"content": [  
  {  
    "id": 1,  
    "nomeProduto": "Açaí"  
  },  
  {  
    "id": 2,  
    "nomeProduto": "Cuscuz"  
  },  
  {  
    "id": 3,  
    "nomeProduto": "Bolo de mandioca"  
  },  
  {  
    "id": 4,  
    "nomeProduto": "Carne de sol"  
  }  
],  
"pageable": {  
  "pageNumber": 0,  
  "pageSize": 20,  
  "sort": {  
    "empty": true,  
    "sorted": false,  
    "unsorted": true  
  },  
}
```

```
        "offset": 0,
        "unpaged": false,
        "paged": true
    },
    "last": true,
    "totalElements": 4,
    "totalPages": 1,
    "first": true,
    "size": 20,
    "number": 0,
    "sort": {
        "empty": true,
        "sorted": false,
        "unsorted": true
    },
    "numberOfElements": 4,
    "empty": false
}
```

Status code: 200, 403.

Erro (403):{

```
    "timestamp": "2026-06-06T14:34:47.9554476",
    "status": 403,
    "erro": "Acesso negado",
    "mensagem": "Você não tem permissão para acessar este recurso",
    "path": "/produtos"
}
```

4.8 /auditorias - Módulo de auditoria de ações sensíveis

/auditorias - Listar

Finalidade: Consultar ações sensíveis registradas.

Parâmetros:

Query:

- page - Número da página

- size - Quantidade de registros
- sort - Ordenação dos resultados

Método: GET

Rota: /auditorias

Autenticação e permissões: JWT obrigatório. Perfil ADMIN.

Request: Não possui corpo de requisição.

Response: {

```
  "content": [  
    {  
      "id": 1,  
      "acao": "CRIAR_PEDIDO",  
      "detalhes": "Pedido criado para usuário 1",  
      "usuario": "Joao da Silva",  
      "data": "2026-06-02T17:31:23.589625"  
    }  
  ],  
  "pageable": {  
    "pageNumber": 0,  
    "pageSize": 20,  
    "sort": {  
      "empty": true,  
      "sorted": false,  
      "unsorted": true  
    },  
    "offset": 0,  
    "paged": true,  
    "unpaged": false  
  },  
  "last": true,  
  "totalElements": 1,  
  "totalPages": 1,  
  "first": true,  
  "numberOfElements": 1,  
  "size": 20,
```

```

    "number": 0,
    "sort": {
        "empty": true,
        "sorted": false,
        "unsorted": true
    },
    "empty": false
}

```

Status Code: 200, 401, 403.

Erro (401):{

```

    "timestamp": "2026-06-06T18:53:57.9838081",
    "status": 401,
    "erro": "Não autorizado",
    "mensagem": "Autenticação necessária para acessar este recurso",
    "path": "/auditorias"
}

```

5 ENTREGA TÉCNICA

[Repositório](#) do Github.

Após inicializar a aplicação, o swagger está disponível em:

<http://localhost:8080/swagger-ui/index.html>

A coleção de testes Insomnia/Postman está disponível no repositório do projeto, em
/docs/raizes-do-nordeste.postman_collection.json

6 PLANOS DE TESTES E EVIDÊNCIAS

Quadro 7 - Plano de testes e evidências Insomnia/Postman

ID	Cenário	Endpoint	Pré-Condição	Entrada	Espera do	Evidência
T01	Login Válido	POST /auth/login	Usuário cadastrado	{email, senha}	200 + token	Login válido
T02	Acesso sem Token	GET /auditorias	—	—	401 + JSON padrão	Listar sem autenticação
T03	Acesso com	GET	—	—	403 +	Listar sem

	perfil sem permissão	/auditorias			JSON padrão	autorização
T04	Campo obrigatório ausente	POST /pedidos	Usuário logado (utilizar Token) Perfil CLIENTE	—	400 + JSON padrão	Criar pedido com campo ausente
T05	Tipo/Formato inválido	POST /pedidos	Usuário logado (utilizar Token) Perfil CLIENTE	{unidadeId, canalPedido, metodoPagamento, itens:[{produtoId, quantidade}] }	400 + JSON padrão	Criar pedido com canalPedido inválido
T06	Pedido com itens válidos	POST /pedidos	Usuário logado (utilizar Token) Perfil CLIENTE	{unidadeId, canalPedido, metodoPagamento, itens:[{produtoId, quantidade}] }	201 + JSON padrão	Criar pedido válido
T07	Pedido com produto/unidade inexistente	POST /pedidos	Usuário logado (utilizar Token) Perfil CLIENTE	{unidadeId, canalPedido, metodoPagamento, itens:[{produtoId, quantidade}] }	404 + JSON padrão	Criar pedido com unidade inexistente
T08	Pedido com estoque insuficiente	POST /pedidos	Usuário logado (utilizar Token) Perfil CLIENTE	{unidadeId, canalPedido, metodoPagamento, itens:[{produtoId, quantidade}] }	409 + JSON padrão	Criar pedido com estoque insuficiente
T09	Pagamento aprovado	GET pedidos/{id}/status	Usuário logado (utilizar Token) Perfil ADMIN ou CLIENTE, pedido ter sido aprovado no endpoint POST /pagamentos	—	200 + JSON padrão	Consultar status pedido aprovado
T10	Pagamento recusado	GET pedidos/{id}/status	Usuário logado (utilizar Token) Perfil ADMIN	—	200 + JSON padrão	Consultar status pedido

			ou CLIENTE, pedido ter sido recusado no endpoint POST /pagamentos			recusado
T11	Logs/Auditoria	GET /auditorias	Usuário logado (utilizar Token) Perfil ADMIN	—	200 + JSON padrão	Listar válido

Fonte: Autor (2026).

7 CONCLUSÃO

O sistema desenvolvido tem como objetivo apoiar a gestão operacional de uma rede de restaurantes, contemplando o gerenciamento de usuários, unidades, produtos, cardápios, estoque, pedidos, pagamentos e auditoria. O foco principal da implementação foi garantir a integridade do fluxo de pedidos, desde sua criação até a confirmação ou recusa do pagamento.

A modelagem do sistema foi construída a partir dos requisitos de negócio levantados inicialmente. O Diagrama Entidade-Relacionamento (DER) serviu como base para a definição das entidades persistidas no banco de dados e seus respectivos relacionamentos, enquanto o Diagrama de Classes detalhou os atributos, relacionamentos e responsabilidades de cada objeto da aplicação. Os Casos de Uso orientaram a construção dos endpoints REST, permitindo que cada funcionalidade implementada correspondesse diretamente a uma necessidade identificada durante a análise do sistema.

O fluxo crítico do projeto consiste na criação de pedidos. Durante esse processo, o sistema valida a existência do usuário, da unidade e dos produtos selecionados, verifica a disponibilidade do item no cardápio da unidade escolhida, controla restrições sazonais, valida a disponibilidade de estoque e registra a reserva dos itens solicitados. Após a criação do pedido, é gerado um pagamento associado, inicialmente com status pendente.

O pagamento foi implementado por meio de um mecanismo mock, permitindo que usuários com perfil administrativo aprovelem ou recusem pagamentos manualmente.

Quando aprovado, o pedido tem seu status atualizado de forma consistente com o resultado da operação. Da mesma forma, quando recusado, o sistema registra adequadamente a situação, garantindo coerência entre o pagamento e o pedido relacionado.

Em relação à segurança, foram adotadas medidas compatíveis com aplicações web modernas. As senhas são armazenadas utilizando criptografia BCrypt, impedindo o armazenamento de credenciais em texto puro. A autenticação é realizada por meio de JSON Web Token (JWT), enquanto a autorização utiliza controle baseado em perfis (roles), restringindo o acesso às funcionalidades administrativas. Além disso, informações sensíveis como senhas não são expostas nos DTOs de resposta enviados pela API. Atualmente existem dois perfis implementados (ADMIN, CLIENTE), que realizam as principais funções do sistema, mas atualizações posteriores podem ampliar os roles (COZINHA, ATENDENTE, GERENTE, por exemplo) com o intuito de descentralizar funções que hoje são exercidas exclusivamente pelo administrador.

Sob a perspectiva da LGPD, o sistema atualmente solicita apenas dados essenciais para funcionamento da aplicação, limitando-se a nome, e-mail e senha do usuário. A minimização da coleta de dados reduz riscos relacionados ao tratamento de informações pessoais. Futuras evoluções poderão ampliar o conjunto de informações armazenadas para permitir experiências mais personalizadas, sempre respeitando princípios de necessidade e finalidade.

Os mecanismos de validação e tratamento de erros foram padronizados para fornecer respostas consistentes ao consumidor da API. Foram tratados cenários como autenticação inválida, acesso sem permissão, recursos inexistentes, erros de validação de dados, parâmetros inválidos e violações de regras de negócio, utilizando códigos HTTP apropriados e mensagens padronizadas.

Os testes realizados evidenciam o correto funcionamento das funcionalidades implementadas. Foram validados cenários positivos e negativos envolvendo autenticação, autorização, validação de dados, criação de pedidos, controle de estoque, processamento de pagamentos e registro de auditoria. As evidências

encontram-se organizadas na coleção Postman/Insomnia entregue juntamente com o projeto.

Algumas funcionalidades foram parcialmente implementadas ou mantidas como proposta para evolução futura. O sistema de auditoria registra atualmente ações relacionadas à criação e ao cancelamento de pedidos, podendo ser expandido para abranger alterações em produtos, acessos administrativos e outras operações sensíveis. O programa de fidelidade também foi iniciado, permitindo que usuários optem por participar e acumulem pontos, porém o mecanismo completo de utilização desses pontos em benefícios ainda não foi concluído. Da mesma forma, promoções e descontos são atualmente gerenciados manualmente pelos administradores através da configuração de preços sazonais nos cardápios das unidades, podendo futuramente evoluir para campanhas promocionais automatizadas.

Dessa forma, considera-se que os objetivos principais do projeto foram atingidos, priorizando a entrega dos requisitos centrais para o funcionamento do sistema, fornecendo uma API funcional, segura, organizada em camadas.

8 REFERÊNCIAS

LUCIDCHART. **O que é diagrama de classe UML?** Disponível em: <https://www.lucidchart.com/pages/pt/o-que-e-diagrama-de-classe-uml>. Acesso em: 23 mar. 2026.

LUCIDCHART. **Diagrama de caso de uso UML.** Disponível em: <https://www.lucidchart.com/pages/pt/diagrama-de-caso-de-uso-uml>. Acesso em: 23 mar. 2026.

LUCIDCHART. **O que é diagrama entidade-relacionamento (DER)?** Disponível em: <https://www.lucidchart.com/pages/pt/o-que-e-diagrama-entidade-relacionament> o. Acesso em: 23 mar. 2026.

SPRING. **Spring Boot.** Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: 5 maio. 2026.

BAELDUNG. **Spring Data REST Relationships**. Disponível em:
<https://www.baeldung.com/spring-data-rest-relationships>. Acesso em: 5 maio
2026.